

Types of exception in java :

We have 3 kinds of exception

- 1) Checked Exception
- 2) Runtime Exception
- 3) Error (Serious Problem and Developer is not responsible)
- ➡ Unchecked Exception

***Difference between Checked Exception and Unchecked Exception :

Checked Exception :-

Checked exceptions are compile-time exceptions that must be handled (try-catch) or declared (throws) by the programmer so, at runtime we will not face any problem during execution.

Here compiler takes very much care and wanted the clarity regarding the exception by saying that, by using this code you may face some problem at runtime and you did not report me how would you handle this situation at runtime so, provide either try-catch or declare the method as throws.

Except RuntimeException, all the checked exceptions are directly sub class of java.lang.Exception OR java.lang.Throwable (except Error).

Example :

FileNotFoundException, IOException, InterruptedException, ClassNotFoundException, SQLException, CloneNotSupportedException, EOFException and so on

Unchecked Exception :-

An unchecked exception is an exception that occurs at runtime and is not checked by the compiler, so it does not need to handle by using try-catch or using throws keyword.

Unchecked exceptions are directly entertain by JVM because they are rarely occurred in java.[And known as RuntimeException]

Error, sub classes of Error, RuntimeException and its sub classes are Unchecked Exception.

Example :

RuntimeException, ArithmeticException, ArrayIndexOutOfBoundsException, NullPointerException, NumberFormatException, ClassCastException, ArrayStoreException and so on.

Some Bullet points regarding Checked and Unchecked :

- Checked Exception :

1) Compiler takes care (Will not compile the code)
2) Handling is compulsory (try-catch OR throws)
3) Directly the sub class of java.lang.Exception OR java.lang.Throwable (Excluding RuntimeException and java.lang.Error)
- Unchecked Exception :

1) Comiler will not take any care (Will compile the code)
2) Handling is Optional. (Developer choice)
3) Error and its sub classes, RuntimeException and its sub classes

***Why compiler takes very much care regarding the checked Exception ?**

As we know Checked Exceptions are very common exception so in case of checked exception "handling is compulsory" (The code should be terminated normally) because checked Exception depends upon other resources as shown below.

- IOException (we are depending upon System Keyboard OR Files)
- FileNotFoundException(We are depending upon the file)
- InterruptedException (Thread related problem)
- ClassNotFoundException (class related problem)
- SQLException (SQL related or database related problem)
- CloneNotSupportedException (Object is the resource)
- EOFException(We are depending upon the file)

When to provide try-catch or declare the method as throws for Checked Exception :-

try-catch

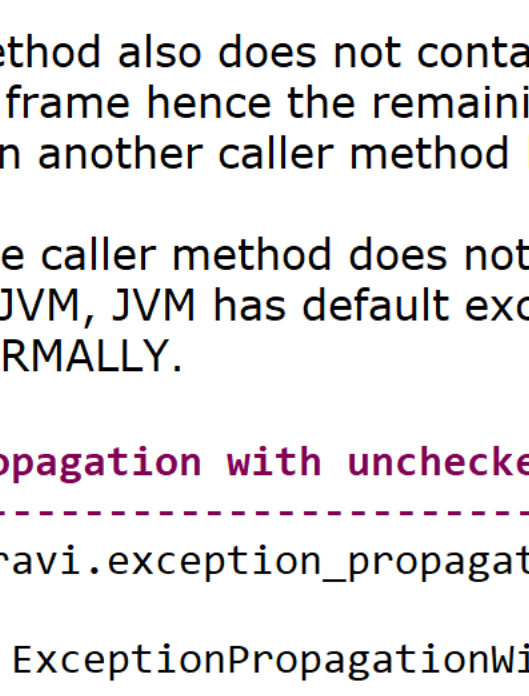
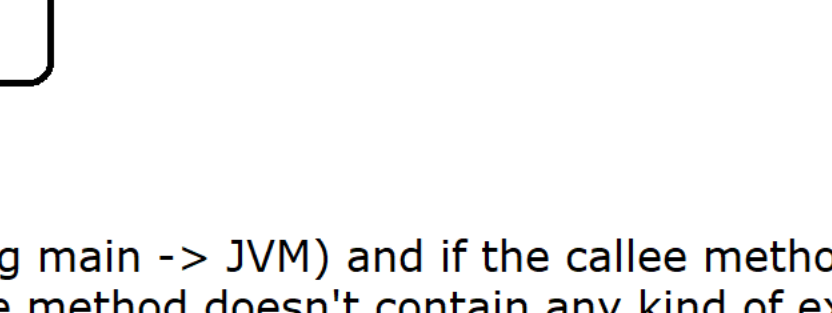
We should provide try-catch if we want to handle the exception in the method where checked exception is encountered, as well as if we want to provide user-defined message to the client.

throws :

throws keyword describes that the method might throw an Exception, It also might not (Exception always encounter at runtime). It is used only at the end of a method declaration to indicate what exceptions it supports OR what type of Exception it might throw which will be handled by JVM OR caller method.

Note :- It is always better to use try catch so we can provide appropriate user defined messages to our client. In case of throws, If an exception encounter at runtime then remaining lines of code will not be executed.

Exception propagation [Propagation of Exception Object from Callee to Caller method]



In m2() method, **AE is encountered** but m2() method is not interested to handle the exception (not writing try-catch) so now JVM will terminate the m2() method and AE object will be propagated to the caller method i.e m1() method

If any of the caller metod is not defining try-catch to handle the exception then at last exception will be propagated to the JVM default exception handler

default exception handler will print the exception details and terminate the programs **ab-normally**

Whenever we call a method (including main -> JVM) and if the callee method contains any kind of exception (checked OR Unchecked) and if callee method doesn't contain any kind of exception handling mechanism (try-catch) then JVM will propagate the exception object to caller method for handling purpose. This is called Exception Propagation.

If the caller method also does not contain any exception handling mechanism then JVM will terminate the method from the stack frame hence the remaining part of the method(m1 method) will not be executed even if we handle the exception in another caller method like main.

If any of the the caller method does not contain any exception handling mechanism (try-catch) then exception will be handled by JVM, JVM has default exception handler which will provide the exception message and terminates the program ABNORMALLY.

```
Exception Propagation with unchecked
-----
package com.ravi.exception_propagation;

public class ExceptionPropagationWithUnchecked
{
    public static void main(String[] args)
    {
        IO.println("main method started");
        try
        {
            m1();
        }
        catch(ArithmeticException e)
        {
            IO.println("Handled by main method");
        }
        IO.println("main method ended");
    }

    public static void m1()
    {
        IO.println("m1 method started");
        m2();
        IO.println("m1 method ended");
    }

    public static void m2()
    {
        IO.println("m2 method started");

        IO.println(10/0);    //AE

        IO.println("m2 method ended");
    }
}
```

```
Exception Propagation with checked
-----
package com.ravi.exception_propagation;

class Foo
{
    static
    {
        IO.println("Static Block");
    }
}

public class ExceptionPropagationWithChecked
{
    public static void main(String[] args)
    {
        IO.println("main method started");
        m1();
        IO.println("main method ended");
    }

    public static void m1()
    {
        IO.println("m1 method started");
        try
        {
            m2();
        }
        catch(ClassNotFoundException e)
        {
            IO.println("Handled by m1 method");
        }
        IO.println("m1 method ended");
    }

    public static void m2() throws ClassNotFoundException
    {
        IO.println("m2 method started");

        Class.forName("com.ravi.exception_propagation.Foo");

        IO.println("m2 method ended");
    }
}
```